

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 7
По дисциплине «Современные платформы программирования»

Выполнил:
Шевчук А. В.
Студент группы ПО-13
Проверил:
Кулик А. Д.

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Ход работы:

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

7) Изобразить в окне приложения отрезок, вращающийся в плоскости формы вокруг точки, движущейся по отрезку.

1.py:

```
"""
Отрезок вращается вокруг точки, движущейся по другому отрезку.
"""

from dataclasses import dataclass
from datetime import datetime
from math import cos, radians, sin
from pathlib import Path
import tkinter as tk

@dataclass
class Point:
    """Точка на плоскости."""
    x: float
    y: float

def rotate_point(point: Point, center: Point, angle_degrees: float) -> Point:
    """Поворачивает точку вокруг центра на заданный угол."""
    angle = radians(angle_degrees)
    dx = point.x - center.x
    dy = point.y - center.y

    return Point(
        center.x + dx * cos(angle) - dy * sin(angle),
        center.y + dx * sin(angle) + dy * cos(angle),
    )

def point_on_segment(start: Point, end: Point, coefficient: float) -> Point:
    """Возвращает точку на отрезке по коэффициенту от 0 до 1."""
    return Point(
        start.x + (end.x - start.x) * coefficient,
        start.y + (end.y - start.y) * coefficient,
    )

def segment_end(pivot: Point, length: float, angle_degrees: float) -> Point:
    """Возвращает конец вращающегося отрезка."""
    start_position = Point(pivot.x + length, pivot.y)
    return rotate_point(start_position, pivot, angle_degrees)

def changing_color(angle_degrees: float) -> str:
    """Возвращает цвет, изменяющийся при вращении."""
    red = int((sin(radians(angle_degrees)) + 1) * 127)
    green = int((sin(radians(angle_degrees + 120)) + 1) * 127)
    blue = int((sin(radians(angle_degrees + 240)) + 1) * 127)

    return f"#{red:02x}{green:02x}{blue:02x}"
```

```

class RotatingSegmentApp:
    """Оконное приложение с анимацией отрезка."""

    def __init__(self) -> None:
        """Создает окно приложения."""
        self.root = tk.Tk()
        self.root.title("ЛР7. Задание 1. Вариант 7")

        self.canvas = tk.Canvas(self.root, width=800, height=500, bg="white")
        self.canvas.pack()

        panel = tk.Frame(self.root)
        panel.pack(pady=5)

        tk.Label(panel, text="Скорость:").grid(row=0, column=0, padx=5)
        self.speed_scale = tk.Scale(panel, from_=1, to=10, orient=tk.HORIZONTAL)
        self.speed_scale.set(4)
        self.speed_scale.grid(row=0, column=1, padx=5)

        tk.Label(panel, text="Длина отрезка:").grid(row=0, column=2, padx=5)
        self.length_scale = tk.Scale(panel, from_=50, to=200, orient=tk.HORIZONTAL)
        self.length_scale.set(120)
        self.length_scale.grid(row=0, column=3, padx=5)

        self.pause_button = tk.Button(panel, text="Пауза", command=self.change_pause)
        self.pause_button.grid(row=0, column=4, padx=5)

        self.screenshot_button = tk.Button(panel, text="Скриншот", command=self.save_screenshot)
        self.screenshot_button.grid(row=0, column=5, padx=5)

        self.guide_start = Point(120, 250)
        self.guide_end = Point(680, 250)
        self.coefficient = 0.0
        self.direction = 1
        self.angle = 0.0
        self.paused = False

        self.animate()

    def change_pause(self) -> None:
        """Ставит анимацию на паузу или продолжает ее."""
        self.paused = not self.paused
        self.pause_button.config(text="Продолжить" if self.paused else "Пауза")

    def save_screenshot(self) -> None:
        """Сохраняет изображение canvas в текущую папку."""
        file_name = f"task1_screenshot_{datetime.now().strftime('%H%M%S')}.ps"
        self.canvas.postscript(file=str(Path.cwd() / file_name), colormode="color")

    def draw_scene(self) -> None:
        """Рисует текущий кадр."""
        self.canvas.delete("all")

        pivot = point_on_segment(self.guide_start, self.guide_end, self.coefficient)
        length = self.length_scale.get()
        end = segment_end(pivot, length, self.angle)
        color = changing_color(self.angle)

        self.canvas.create_line(
            self.guide_start.x,
            self.guide_start.y,
            self.guide_end.x,
            self.guide_end.y,
            fill="gray",
            dash=(5, 3),
            width=2,
        )
        self.canvas.create_oval(pivot.x - 6, pivot.y - 6, pivot.x + 6, pivot.y + 6, fill="black")
        self.canvas.create_line(pivot.x, pivot.y, end.x, end.y, fill=color, width=5)

        self.canvas.create_text(
            400,
            30,
            text="Отрезок вращается вокруг точки, движущейся по отрезку",
            font=("Arial", 14),
        )

    def animate(self) -> None:
        """Выполняет анимацию."""
        if not self.paused:
            speed = self.speed_scale.get()

```

```

        self.angle += speed * 3
        self.coefficient += self.direction * speed * 0.0015

        if self.coefficient >= 1:
            self.coefficient = 1
            self.direction = -1

        if self.coefficient <= 0:
            self.coefficient = 0
            self.direction = 1

    def draw_scene():
        self.root.after(30, self.animate)

    def run(self) -> None:
        """Запускает приложение."""
        self.root.mainloop()

if __name__ == "__main__":
    RotatingSegmentApp().run()

```



Задание 2: Реализовать построение заданного типа фрактала по варианту Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

2.ру:

```

"""
Построение фрактала: снежинка Коха.
"""

from dataclasses import dataclass
from datetime import datetime
from math import cos, radians, sin, sqrt
from pathlib import Path
import tkinter as tk

@dataclass
class Point:
    """Точка на плоскости."""

    x: float
    y: float

def rotate_point(point: Point, center: Point, angle_degrees: float) -> Point:
    """Поворачивает точку вокруг центра."""
    angle = radians(angle_degrees)
    dx = point.x - center.x
    dy = point.y - center.y

```

```

    return Point(
        center.x + dx * cos(angle) - dy * sin(angle),
        center.y + dx * sin(angle) + dy * cos(angle),
    )

def divide_segment(start: Point, end: Point) -> tuple[Point, Point]:
    """Возвращает точки, делящие отрезок на три части."""
    first = Point(
        start.x + (end.x - start.x) / 3,
        start.y + (end.y - start.y) / 3,
    )
    second = Point(
        start.x + 2 * (end.x - start.x) / 3,
        start.y + 2 * (end.y - start.y) / 3,
    )

    return first, second

def koch_curve(start: Point, end: Point, depth: int) -> list[Point]:
    """Строит точки кривой Коха."""
    if depth == 0:
        return [start, end]

    first, second = divide_segment(start, end)
    peak = rotate_point(second, first, -60)

    points = []
    points.extend(koch_curve(start, first, depth - 1)[::-1])
    points.extend(koch_curve(first, peak, depth - 1)[::-1])
    points.extend(koch_curve(peak, second, depth - 1)[::-1])
    points.extend(koch_curve(second, end, depth - 1))

    return points

def koch_snowflake_points(center: Point, size: float, depth: int) -> list[Point]:
    """Возвращает точки снежинки Коха."""
    height = size * sqrt(3) / 2

    first = Point(center.x - size / 2, center.y + height / 3)
    second = Point(center.x + size / 2, center.y + height / 3)
    third = Point(center.x, center.y - 2 * height / 3)

    result = []

    for start, end in [(first, second), (second, third), (third, first)]:
        curve = koch_curve(start, end, depth)

        if result:
            result.extend(curve[1:])
        else:
            result.extend(curve)

    return result

class KochSnowflakeApp:
    """Оконное приложение для построения снежинки Коха."""

    def __init__(self) -> None:
        """Создает окно приложения."""
        self.root = tk.Tk()
        self.root.title("ЛР7. Задание 2. Снежинка Коха")

        self.canvas = tk.Canvas(self.root, width=800, height=600, bg="white")
        self.canvas.pack()

        panel = tk.Frame(self.root)
        panel.pack(pady=5)

        tk.Label(panel, text="Глубина:").grid(row=0, column=0, padx=5)
        self.depth_spinbox = tk.Spinbox(panel, from_=0, to=5, width=5)
        self.depth_spinbox.grid(row=0, column=1, padx=5)
        self.depth_spinbox.delete(0, tk.END)
        self.depth_spinbox.insert(0, "3")

        tk.Label(panel, text="Размер:").grid(row=0, column=2, padx=5)
        self.size_scale = tk.Scale(panel, from_=100, to=450, orient=tk.HORIZONTAL)
        self.size_scale.set(350)
        self.size_scale.grid(row=0, column=3, padx=5)

        tk.Button(panel, text="Построить", command=self.draw_fractal).grid(row=0, column=4,
padx=5)
        tk.Button(panel, text="Очистить", command=self.clear_canvas).grid(row=0, column=5,
padx=5)
        tk.Button(panel, text="Скриншот", command=self.save_screenshot).grid(row=0, column=6,
padx=5)

```

```

        self.draw_fractal()

def clear_canvas(self) -> None:
    """Очищает холст."""
    self.canvas.delete("all")

def save_screenshot(self) -> None:
    """Сохраняет изображение canvas в текущую папку."""
    file_name = f"task2_screenshot_{datetime.now().strftime('%H_%M_%S')}.ps"
    self.canvas.postscript(file=str(Path.cwd() / file_name), colormode="color")

def draw_fractal(self) -> None:
    """Рисует снежинку Коха."""
    self.canvas.delete("all")

    depth = int(self.depth_spinbox.get())
    size = self.size_scale.get()
    points = koch_snowflake_points(Point(400, 320), size, depth)

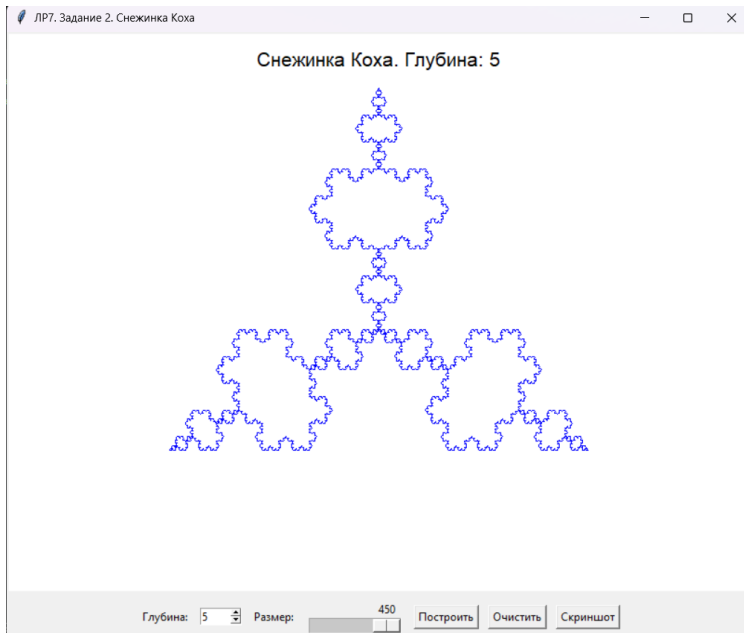
    for index in range(len(points) - 1):
        start = points[index]
        end = points[index + 1]
        self.canvas.create_line(start.x, start.y, end.x, end.y, fill="blue", width=1)

    self.canvas.create_text(
        400,
        30,
        text=f"Снежинка Коха. Глубина: {depth}",
        font=("Arial", 16),
    )

def run(self) -> None:
    """Запускает приложение."""
    self.root.mainloop()

if __name__ == "__main__":
    KochSnowflakeApp().run()

```



Вывод работы: освоил возможности языка программирования Python в разработке оконных приложений.